ApplePie

A Complete Technical Breakdown

Repository: https://github.com/UserJoo9/ApplePie

Activation Lock Bypass via Python + 3uTools

This document explains every layer of the ApplePie project: what it is, how it works, how Python and 3uTools interact, how iPhone internal files are accessed, the full build structure, requirements, pros, cons, and limitations.

1. What is ApplePie?

ApplePie is an open-source, community-built tool that bypasses Apple's Activation Lock (formerly 'Find My iPhone / Find My iPad') without requiring the original Apple ID and password. It is designed for scenarios such as:

- Buying a used iPhone/iPad that is still linked to the seller's iCloud account.
- Inheriting a device whose owner has passed away.
- Corporate devices where the admin's Apple ID is no longer accessible.

The tool does NOT jailbreak the device and does NOT require a paid Apple server-side unlock. Instead, it manipulates the device's local activation state files directly, tricking iOS into believing the activation lock has already been satisfied.

2. High-Level Architecture

The project is a thin orchestration layer sitting on top of two main components:

1. Python Scripts – The 'brain'. They parse user input, validate the device state, locate the correct plist files, modify XML/plist keys, and trigger 3uTools via its command-line or file-system interface.

2. 3uTools (3u) – The 'hands'. A Windows desktop application (originally by Chuhai/3u) that speaks Apple's proprietary USB protocols to read and write files inside the iPhone's sandboxed file system, flash firmware, and manage the device in DFU / Recovery modes.

Together they form a pipeline:
User → Python CLI → 3uTools USB bridge → iPhone file system → modified plist → reboot → unlocked device.

3. Repository Structure

The repo is intentionally small and focused. A typical layout:

```
ApplePie/
├── README.md # Full instructions, screenshots, credits
├── applepie.py # Main entry-point script
├── plist_utils.py # Helper: parse / write .plist (XML & binary)
├── device_info.py # Read ECID, UDID, iOS version via 3uTools
├── bypass/
│ ├── findmy.py # Core bypass logic for Find My / Activation Lock
│ ├── icloud_prefs.py # Read/write com.apple.icloud.* plist files
│ └── activation.py # Handle com.apple.mobileactivationd.plist
├── 3utools_bridge.py # Wrapper that shells out to 3uTools CLI
├── requirements.txt # Python deps (plistlib is stdlib, minimal extras)
├── assets/
│ ├── screenshots/ # Step-by-step UI screenshots
│ └── diagrams/ # Architecture / flow diagrams
├── LICENSE # MIT
└── .gitignore
```

Key observations:

- No compiled binaries are shipped – everything is pure Python.
- 3uTools is NOT bundled; the user must install it separately.
- The `bypass/` package isolates the file-manipulation logic so it can be unit-tested independently of the USB layer.
- `plist_utils.py` handles both Apple's XML plist format and the binary plist format (bplist00) that iOS uses on-disk.

4. How the Bypass Works – Conceptual Flow

When an iPhone is Activation-Locked, iOS stores the lock state in several files under /var/mobile/Library/Preferences/. The most critical ones are:

• com.apple.icloud.findmydevice.plist
– Contains the 'FMAcknowledgedVersions' array, the associated Apple ID (as a token), and a boolean 'Enabled' key.

• com.apple.mobileactivationd.plist
– Stores the 'ActivationState' key (values like 'ActivationStateActivated', 'ActivationStateAwaitingActivation', etc.) and the 'ActivationRecord' dictionary.

• com.apple.icloud.preferences.plist
– Holds the iCloud account association and the 'FindMy' toggle.

ApplePie's strategy:

Step 1 – Connect: The device is plugged into a Windows PC. 3uTools detects it over USB using Apple's ECID (Exclusive Chip ID) and UDID.

Step 2 – Mount / Access: 3uTools opens a file-system session. On non-jailbroken devices it uses the AFC (Apple File Conduit) protocol exposed through the Lockdown port, or in DFU mode it can write directly to the NAND partition.

Step 3 – Read: The Python script calls 3uTools to pull the relevant plist files into a local temp folder.

Step 4 – Modify: Python parses the plist (XML or binary), edits the critical keys:
– Sets 'ActivationState' → 'ActivationStateActivated'
– Clears or removes 'FMAcknowledgedVersions'
– Sets 'Enabled' → NO in findmydevice.plist
– Removes or nulls the 'ActivationRecord' dict

Step 5 – Write Back: The modified plists are pushed back to the device via 3uTools.

Step 6 – Reboot: The device is told to restart. On boot, iOS reads the activation state, sees 'Activated', and skips the Activation Lock screen.

Step 7 – Verify: The Python script re-reads the files to confirm the changes stuck and prints a success/failure message.

5. The Python Component in Detail

5.1 applepie.py – Main Entry Point

This is the CLI the user interacts with. It typically:
- Prompts for the 3uTools installation path (or auto-detects it).
- Asks the user to connect the iPhone and trust the computer.
- Calls device_info.py to grab ECID, UDID, iOS version, model.
- Runs a pre-flight check (iOS version within supported range, battery > 50 %, Find My enabled, etc.).
- Orchestrates the bypass/ package.
- Prints a summary and a warning that data may be lost.

5.2 plist_utils.py – Plist Handling

Apple stores preferences as plist files. Two formats exist:

- XML plist (<plist version="1.0"> … </plist>)
- Binary plist (magic bytes bplist00)

This module wraps Python's built-in plistlib module and adds:
- Auto-detection of format by reading the first 8 bytes.
- Safe read → modify → write cycle that preserves file permissions (root:wheel, 0644).
- A 'diff' helper that shows exactly which keys changed before writing back, so the user can confirm.

Example (simplified):

```
import plistlib

def load_plist(path: str):
 with open(path, 'rb') as f:
 raw = f.read()
 if raw[:8] == b'bplist00':
 return plistlib.loads(raw) # binary
 return plistlib.loads(raw) # XML (same API)

def save_plist(path: str, data: dict):
 with open(path, 'wb') as f:
 f.write(plistlib.dumps(data))
 os.chmod(path, 0o644)
```

5.3 bypass/findmy.py – Core Bypass Logic

This is the heart of the project. It:

1. Locates /var/mobile/Library/Preferences/com.apple.icloud.findmydevice.plist on the device (via the 3uTools bridge).
2. Reads the plist.
3. Sets the 'Enabled' key to False (or removes it).

4. Clears the 'FMAcknowledgedVersions' list so iOS doesn't re-trigger the lock on next iCloud sign-in.
5. Writes the modified plist back.

It also handles edge cases:
• Device already has Find My disabled (no-op).
• plist file not found (older iOS versions store it elsewhere).
• File permission errors (needs root via 3uTools' 'root' toggle).

5.4 bypass/activation.py – Activation State

Handles com.apple.mobileactivationd.plist:
• Reads 'ActivationState' and 'ActivationRecord'.
• Overwrites 'ActivationState' with 'ActivationStateActivated'.
• Removes the 'ActivationRecord' dictionary (which contains the server-issued activation ticket).
• Optionally clears 'ActivationTime' so iOS doesn't re-query Apple's activation servers immediately.

On some iOS versions the key lives in a different plist (com.apple.lic.plist or the 'SetupAssistant' plist). The script tries multiple paths and uses the first one that exists.

6. The 3uTools Component in Detail

6.1 What is 3uTools?

3uTools (also written '3u') is a free Windows application developed by the Chinese company Chuhai. It is one of the most popular iOS management tools in Asia and increasingly in the West. Its relevant capabilities for ApplePie are:

- USB device detection via Apple's proprietary protocol.
- 'File Management' / 'File Explorer' – browse, copy, delete files on the device.
- 'System Repair' / 'Firmware Flash' – put device in DFU/Recovery and write IPSW firmware.
- 'Root' toggle – on supported iOS versions, grant the tool root-level file access.
- Command-line interface (3uTools.exe with flags) that ApplePie's Python wrapper calls.

6.2 How 3uTools Accesses iPhone Internal Files

This is the most critical question. iOS sandboxes its file system heavily. 3uTools gets around this through several mechanisms depending on the device state:

A) Lockdown + AFC (Apple File Conduit) Protocol
– When the iPhone is booted and 'Trusted' is tapped on the device, it opens a USB port (typically 62078) that speaks the Lockdown protocol. Over Lockdown, 3uTools can open an AFC session.
– AFC normally only exposes /var/mobile/Media/ (photos, music). However, 3uTools can request additional AFC mounts or use a 'full filesystem' AFC that Apple exposes in certain firmware versions or when the device is in a partial-restore state.

B) DFU / Recovery Mode + NAND Write
– In DFU mode the iPhone's CPU runs a minimal bootloader (iBSS/iBEC). 3uTools can write directly to the NAND flash partitions, including the 'Data' partition where /var/mobile/ lives.
– This is the most reliable method but is also the most destructive (can wipe data if done wrong).

C) 'Root' Mode via MobileDevice / MobileFileTransport
– On iOS ≤ 15 (and some iOS 16 builds), 3uTools can exploit the MobileFileTransport service to gain root-level read/write to the entire /var/mobile hierarchy without DFU.
– On iOS 16+ Apple tightened this, so 3uTools may need the device to be in Recovery mode.

D) Apple Configurator / idevice* Protocol Shims
– 3uTools bundles (or shells out to) the libimobiledevice protocol stack. The Python script can also call `idevicebackup2`, `ideviceinfo`, or `idevicepair` as fallbacks.

In all cases the physical path on the iPhone is:
/var/mobile/Library/Preferences/
/var/mobile/Library/Preferences/com.apple.icloud.findmydevice.plist
/var/mobile/Library/Preferences/com.apple.mobileactivationd.plist
/var/mobile/Library/Preferences/com.apple.icloud.preferences.plist

6.3 3utools_bridge.py – The Python ↔ 3uTools Glue

This module wraps 3uTools' command-line or COM interface:

- Detects the 3uTools install directory (registry key or common paths like C:\Program Files\3uTools\).
- Calls `3uTools.exe --list-devices` to confirm the iPhone is connected.
- Calls `3uTools.exe --file-read <device_path> <local_path>` to pull a plist.
- Calls `3uTools.exe --file-write <local_path> <device_path>` to push the modified plist back.
- Calls `3uTools.exe --reboot` or `--reboot-to-dfu` as needed.
- Handles the 'Trust this Computer' prompt by timing the USB re-enumeration.

Example (simplified):

```
import subprocess, shutil, os

def find_3utools():
 candidates = [
 r'C:\Program Files\3uTools\3uTools.exe',
 r'C:\Program Files (x86)\3uTools\3uTools.exe',
 os.path.expanduser(r'~\3uTools\3uTools.exe'),
 ]
 for c in candidates:
 if os.path.isfile(c):
 return c
 return shutil.which('3uTools')

def read_device_file(exe, device_path, local_path):
 cmd = [exe, '--file-read', device_path, local_path]
 subprocess.run(cmd, check=True, capture_output=True)

def write_device_file(exe, local_path, device_path):
 cmd = [exe, '--file-write', local_path, device_path]
 subprocess.run(cmd, check=True, capture_output=True)
```

7. Step-by-Step User Walkthrough

Step 1: Install 3uTools

Download the latest 3uTools from the official site, install on Windows 10/11. Make sure iTunes drivers (Apple Mobile Device USB Driver) are present – 3uTools usually installs them automatically.

Step 2: Install Python + deps

Python 3.9+. Run `pip install -r requirements.txt`. Dependencies are minimal: usually just `pyobjc` (macOS fallback) or `psutil` for process checks. plistlib is in the stdlib.

Step 3: Connect iPhone

Plug the iPhone into the PC via USB. Tap 'Trust This Computer' on the phone. Confirm 3uTools shows the device in its main window.

Step 4: Run applepie.py

Open a terminal in the repo folder. Run `python applepie.py`. The script auto-detects 3uTools, reads device info, and prints a pre-flight checklist.

Step 5: Pre-flight checks

The script verifies: iOS version in supported range (typically iOS 11–16), battery > 50 %, Find My is currently ON (so there's something to bypass), no passcode lock interfering.

Step 6: Read plists

3utools_bridge pulls the three plist files to a local temp directory. The user sees a confirmation that files were read successfully.

Step 7: Modify plists

bypass/findmy.py and bypass/activation.py edit the keys. A diff is printed so the user can see exactly what changed.

Step 8: Write plists back

Modified files are pushed to the device. Permissions are set to root:wheel 0644.

Step 9: Reboot

3uTools triggers a reboot. The phone restarts and, because the activation state now reads 'Activated', it boots straight to the home screen (or Setup Assistant without the iCloud gate).

Step 10: Verify

The script re-reads the plists to confirm the changes persisted. The user is told to sign in with their own Apple ID or create a new one.

8. Full Requirements

Hardware

• Windows PC (Windows 10 64-bit or 11 recommended; 3uTools is Windows-only).
• Lightning or USB-C cable (Apple-certified preferred; cheap cables cause USB bus errors that break the Lockdown session).
• iPhone / iPad / iPod Touch with iOS 11 through iOS 16 (sweet spot iOS 12–15). iOS 17+ support is experimental / community-patched.
• iPhone must be charged to at least 50 % to prevent a shutdown mid-write.

Software

• 3uTools – latest version (≥ 4.x). Free download.
• Python 3.9+ (64-bit).
• Apple Mobile Device USB Driver (bundled with 3uTools or iTunes).
• Windows 10/11 with USB 2.0 or 3.0 ports (USB 3.0 preferred).
• Antivirus temporarily disabled (Windows Defender can quarantine 3uTools' helper DLLs).

State of the Device

• Device must be able to show the 'Trust This Computer' dialog (i.e., not in a passcode-locked state that blocks USB access, unless 3uTools' 'passcode bypass' is used).
• Find My / Activation Lock must be currently ACTIVE (otherwise there is nothing to bypass).
• No MDM (Mobile Device Management) profile that re-applies the lock on next iCloud sync.

9. Pros and Cons of This Bypass Method

Pros

• No original Apple ID / password needed – the local file edit is sufficient to clear the lock.
• No jailbreak required – works on stock iOS.
• Free – both Python and 3uTools are free.
• Fast – the whole process takes 5–15 minutes once you know the steps.
• Non-destructive to user data (photos, apps) when using the Lockdown/AFC path instead of DFU flash.
• Open source – you can audit exactly which files are touched.
• Works on a wide iOS range (11–16) with the same codebase.